$F2 \parallel WC_{\max}$ Structural Properties and Dynamic Programming

Author One[a,*], Author Two[b]

[a]Institution One, Location, Country

[b]Institution Two, Location, Country

Abstract

Minimizing $WC_{\max}$ on a two-machine flow shop, denoted $F2 \parallel WC_{\max}$, is a classical NP-hard scheduling problem. Existing exact methods rely on enumerative techniques that scale poorly with instance size, while approximation algorithms often impose restrictive assumptions on processing times. We first sequence all jobs globally by Johnson's rule, obtaining an initial schedule π. Scanning π from left to right, we partition it into at most K blocks: whenever a weight not seen before appears, it serves as the final job of the current block, and the next job starts a new block. We establish structural properties of optimal schedules: the permutation property, the positional constraint that block $\mathcal{G}_k$ jobs cannot appear beyond the first k blocks, and the block-end condition requiring the final job of each block to carry that block's weight (automatically satisfied by construction). The intra-block Johnson ordering inherited from π is further formalized within the DP framework. Building on these properties and the interval-based dynamic programming framework (Hall, 1994) developed for $F2|r_j|C_{\max}$, we design a polynomial-time dynamic programming algorithm with $O(|Y|_{\max} \cdot n)$ complexity that appends jobs block by block. We further show that geometric rounding of weights and processing times converts this algorithm into a polynomial-time approximation scheme.

Keywords: Flow shop scheduling; $WC_{\max}$; Dynamic programming; Structural properties

*Corresponding author. E-mail address: author@email.com

1 Introduction

2 Preliminaries

2.1 Problem Description

We consider the problem $F2 \parallel WC_{\max}$. Let $\mathcal{J} = \{J_1, J_2, \ldots, J_n\}$ be the set of n jobs to be processed on two machines M_1 and M_2 in series: each job J_j must be processed first on M_1 for a_j time units and then on M_2 for b_j time units, with no preemption allowed. M_1 and M_2 can process at most one job at a time. Each job J_j has a weight $w_j > 0$. A schedule $\pi = (\pi(1), \pi(2), \ldots, \pi(n))$ specifies the processing order on both machines. For a schedule π, let $A_j = \sum_{h=1}^{j} a_{\pi(h)}$ and $B_j = \sum_{h=1}^{j} b_{\pi(h)}$ be the cumulative M_1 and M_2 completion times after the first j jobs. The objective is to find a schedule π that minimizes $WC_{\max}$.

For $F2 \parallel WC_{\max}$, a permutation schedule in which the job order on M_1 and M_2, Possess an identical processing order for jobs (Baker, 1974). Johnson's rule (Johnson, 1954) sequences all n jobs to minimize the makespan: jobs with $a_j \le b_j$ go first in non-decreasing order of a_j, and the remaining jobs follow in non-increasing order of b_j.

For any scheduling π. Let $w_{(1)}, w_{(2)}, \ldots, w_{(K)}$ be the K distinct weights among all jobs, and $w_{(1)} > w_{(2)} > \cdots > w_{(K)}$ and define $\mathcal{G}_k = \{J_j \in \mathcal{J} \mid w_j = w^{(k)}\}$, $k = 1, \ldots, K$. For each group $\mathcal{G}_k$, let ℓ_k be the index of the last job of $\mathcal{G}_k$ appearing in π, and set $\ell_0 = 0$. The blocks are defined as $\mathcal{B}_k = \{\pi(\ell_{k-1}+1), \ldots, \pi(\ell_k)\}$, $k = 1, \ldots, K$. Thus each block $\mathcal{B}_k$ ends with the last job of $\mathcal{G}_k$, and block weights are non-increasing: $w(\mathcal{B}_1) \ge w(\mathcal{B}_2) \ge \cdots \ge w(\mathcal{B}_K)$.

For ease of explanation, we construct an illustrative schedule; all subsequent examples refer to this same instance.

Example 2.1. *Consider a 6-job instance with sequence $\pi = (J_1, \ldots, J_6)$ and the following (a_j, b_j, w_j) values:*

	J_1	J_2	J_3	J_4	J_5	J_6
a_j	2	5	6	8	4	3
b_j	5	7	9	4	3	2
w_j	2	2	5	2	4	2

The resulting are $\mathcal{G}_1 = \{J_3\}$, $\mathcal{G}_2 = \{J_5\}$, $\mathcal{G}_3 = \{J_1, J_2, J_4, J_6\}$. And $\mathcal{B}_1 = \{J_1, J_2, J_3\}$, $\mathcal{B}_2 = \{J_4, J_5\}$, $\mathcal{B}_3 = \{J_6\}$.

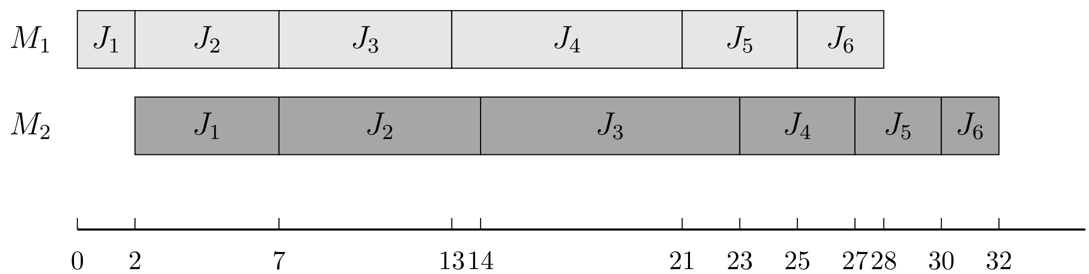

Figure 4.1

Lemma 2.1. *For a given schedule π, rescheduling each blocks to Johnson's ruler(Johnson, 1954) and concatenating blocks schedules delivers a schedule of length at most $WC(\pi)$*

Proof. By construction, the last job of block $\mathcal{B}_k$ carries weight $W(\mathcal{B}_k)$, and every job in $\mathcal{B}_k$ has weight at most $W(\mathcal{B}_k)$, since any weight appearing in $\mathcal{B}_k$ either equals $W(\mathcal{B}_k)$ or has already appeared in an earlier block with larger weight. Let $C^{(k)}_{\max}(\sigma)$ be the completion time of the last job in $\mathcal{B}_k$ under schedule σ. For any job $J_j \in \mathcal{B}_k$, its completion time satisfies $C_j(\sigma) \le C^{(k)}_{\max}(\sigma)$ and $w_j \le W(\mathcal{B}_k)$, hence $w_j C_j(\sigma) \le W(\mathcal{B}_k) \cdot C^{(k)}_{\max}(\sigma)$.

Johnson's rule minimizes the makespan for $F2 \parallel C_{\max}$ (Johnson, 1954). Let π' be the schedule obtained by reordering each $\mathcal{B}_k$ internally by Johnson's rule while preserving the block sequence. We prove $C^{(k)}_{\max}(\pi') \le C^{(k)}_{\max}(\pi)$ for all k by induction on k.

For $k = 1$, block $\mathcal{B}_1$ starts at time zero on both machines under either schedule, so internal Johnson ordering directly gives $C^{(1)}_{\max}(\pi') \le C^{(1)}_{\max}(\pi)$. Assume $C^{(k-1)}_{\max}(\pi') \le C^{(k-1)}_{\max}(\pi)$. On M_1, the start time of $\mathcal{B}_k$ equals the total M_1 work of $\mathcal{B}_1, \dots, \mathcal{B}_{k-1}$, which is unchanged by internal reordering, so $\mathcal{B}_k$ starts at the same time on M_1 under both schedules. On M_2, $\mathcal{B}_k$ starts at max{its own M_1 completion, $C^{(k-1)}_{\max}$}; the first term is identical and the second is no larger under π' by the induction hypothesis. Thus $\mathcal{B}_k$ starts no later on M_2 under π', and with the same M_1 start and internal Johnson ordering, $C^{(k)}_{\max}(\pi') \le C^{(k)}_{\max}(\pi)$.

Therefore $WC_{\max}(\pi') = \max_j w_j C_j(\pi') \le \max_k W(\mathcal{B}_k) \cdot C^{(k)}_{\max}(\pi') \le \max_k W(\mathcal{B}_k) \cdot C^{(k)}_{\max}(\pi) \le WC_{\max}(\pi)$, where the last inequality holds because for the job J^* achieving $WC_{\max}(\pi)$, letting $\mathcal{B}_{k^*}$ be its block, $WC_{\max}(\pi) = w_{J^*} C_{J^*}(\pi) = W(\mathcal{B}_{k^*}) \cdot C^{(k^*)}_{\max}(\pi)$. □

2.2 Mixed Integer Linear Programming Formulation

The problem $F2 \parallel WC_{\max}$ can be formulated as a mixed integer linear program (MILP). For a permutation schedule, let the positions be indexed by $k = 1, \dots, n$. We introduce the following decision variables:

- $x_{jk} \in \{0, 1\}$ — equals 1 if job J_j occupies position k, and 0 otherwise;
- $C^1_k \ge 0$ — completion time of the job at position k on M_1;

- $C_k^2 \geq 0$ — completion time of the job at position k on M_2;
- $C_j \geq 0$ — completion time of job J_j on M_2;
- $Z \geq 0$ — the maximum weighted completion time $WC_{\max}$.
- $M = \sum_{j=1}^{n}(a_j + b_j)$ be a sufficiently large constant.

The MILP model is:

$$\min \quad Z \tag{1}$$

$$\text{s.t.} \quad \sum_{k=1}^{n} x_{jk} = 1, \quad \forall\, j \in \mathcal{J} \tag{2}$$

$$\sum_{j=1}^{n} x_{jk} = 1, \quad \forall\, k = 1, \ldots, n \tag{3}$$

$$C_1^1 = \sum_{j=1}^{n} a_j x_{j1} \tag{4}$$

$$C_k^1 = C_{k-1}^1 + \sum_{j=1}^{n} a_j x_{jk}, \quad \forall\, k = 2, \ldots, n \tag{5}$$

$$C_1^2 = C_1^1 + \sum_{j=1}^{n} b_j x_{j1} \tag{6}$$

$$C_k^2 \geq C_k^1 + \sum_{j=1}^{n} b_j x_{jk}, \quad \forall\, k = 2, \ldots, n \tag{7}$$

$$C_k^2 \geq C_{k-1}^2 + \sum_{j=1}^{n} b_j x_{jk}, \quad \forall\, k = 2, \ldots, n \tag{8}$$

$$C_j \geq C_k^2 - M(1 - x_{jk}), \quad \forall\, j \in \mathcal{J},\ k = 1, \ldots, n \tag{9}$$

$$Z \geq w_j C_j, \quad \forall\, j \in \mathcal{J} \tag{10}$$

$$x_{jk} \in \{0, 1\}, \quad C_k^1, C_k^2, C_j, Z \geq 0 \tag{11}$$

Constraint (2)–(3) ensure a one-to-one assignment between jobs and positions. Constraints (4)–(5) compute the completion time of each position on M_1. Constraints (6)–(8) compute the completion time of each position on M_2: for $k \geq 2$, the job at position k starts on M_2 only after it completes M_1 and the preceding job releases M_2. Constraint (9) links the position-based completion times to the job-based completion times via a big-M formulation: when $x_{jk} = 1$, job J_j is at position k and $C_j \geq C_k^2$; when $x_{jk} = 0$, the right-hand side becomes $C_k^2 - M \leq 0$, which is non-binding. Constraint (10) enforces $Z \geq w_j C_j$ for every job, so that minimizing Z yields the optimal $WC_{\max}$.

3 Problem formulation

Theorem 3.1. $F2 \parallel WC_{\max}$ *is NP-hard.*

Proof. We reduce from the Partition problem: given n positive integers $x_1, x_2, \ldots, x_r$ with $\sum_{i=1}^{r} x_i = 2T$, does there exist subset Z_1 and $Z_2 \subseteq \{1, \ldots, r\}$ such that: $\sum_{i \in Z_1} x_i = \sum_{i \in Z_2} x_i$ $= T$?

Construct an instance of $F2 \parallel WC_{\max}$, Let the number of jobs $n = r + 1$. Let $a_j = 1$, $b_j = rx_j$, $w_j = T + 1$ for jobs $j = 1, \ldots, n - 1$. Put $a_n = rT$, $b_n = 1$, $w_n = 2T + 1$ for job J_n. Does there exist a threshold Y such that:

$$WC_{\max} \leq Y = r(T + 1)(2T + 1)$$

holds?

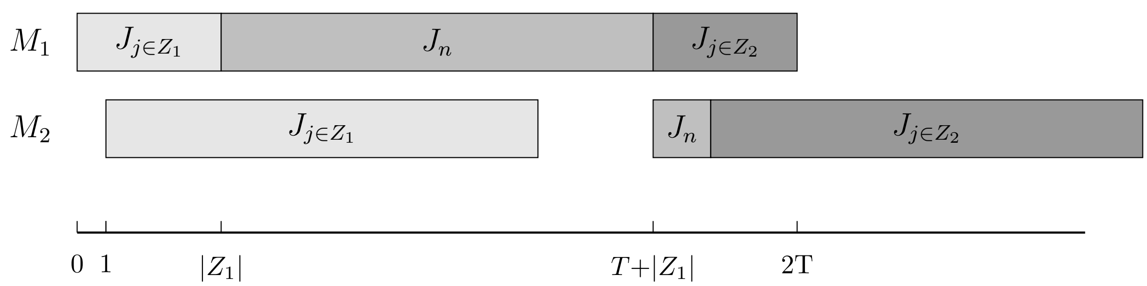

Figure 3-1

We partition jobs j_1 to j_{n-1} into two subsets via j_n, the distribution of jobs on the machine is shown in Figure 3. Since the processing times of A_1 and A_2 are unknown, while their total processing time is 2T. We may assume that the processing time of A_1 on M_2 is $r(T + \xi)$, A_2 is $r(T - \xi)$, where $-T \leq \xi \leq T$. Therefore, the completion time of J_n is determined by the following formula:

$$C_n^1 = |A_1| + rT + 1 \tag{12}$$

$$C_n^2 = 1 + r(T + \xi) + 1 \tag{13}$$

and $C_n = \max\{C_n^1, C_n^2\}$

If $\xi = 0$, then $\sum_{j \epsilon A_1} b_j = \sum_{j \epsilon A_2} b_j$, which means the partition problem has a feasible solution. In this case, the completion time of job J_n is determined by (12), we have:

$$\begin{aligned} C_n &= |A_1| + rT + 1 \leqslant (rT + r) = r(T + 1) \\ w_n C_n &= (2T + 1)(r(T + 1)) = Y \\ WC_{\max} &= r(T + 1)(C_n + rT) = Y \end{aligned}$$

so $w_jC_j \leqslant WC_{\max} = Y$

If the scheduling problem exists a threshold y such that $WC_{\max} \leq y$. When $\xi > 0$, $C_n$is determined by (13), we have:

$$C_n = (1 + r(T + \xi) + 1) \geqslant (1 + r(T + 1) + 1)$$
$$w_nC_n \geqslant (2T + 1)(1 + r(T + 1) + 1) = Y$$

When $\xi < 0$, $C_n$is determined by (12), we have:

$$\begin{aligned} C_{\max} &= (C_n + \sum_{j \epsilon A_2} b_j) \\ &= (1 + rT + |A_1| + r(T - \xi)) \\ &\geqslant (rT + r + rT) \\ &= r(2T + 1) = Y \\ WC_{\max} &\geqslant r(T + 1)(r(2T + 1)) = Y \end{aligned}$$

so $WC_{\max} > Y$, there exists a contradiction.

Therefore, the Partition instance admits a solution if and only if the constructed $F2 \parallel WC_{\max}$ instance admits a schedule with $WC_{\max} \leq Y$. □

Theorem 3.2. *$F2 \parallel WC_{\max}$ is strongly NP-hard.*

Proof. We reduce from the 3-Partition problem: given $3m$ positive integers $x_1, \ldots, x_{3m}$ and a bound B with $B/4 < x_i < B/2$ and $\sum_{i=1}^{3m} x_i = mB$, can the integers be partitioned into m triples each summing to B? Construct an instance of $F2 \parallel WC_{\max}$ with $n = 4m$ jobs as follows (see Figure 3):

- $3m$ element jobs $J_1, \ldots, J_{3m}$: $a_j = 0$, $b_j = x_j$, $w_j = 1$;
- m separator jobs $S_1, \ldots, S_m$: $a_{s_k} = B$, $b_{s_k} = 0$, $w_{s_k} = 3m/k$.

Element jobs have $a_j = 0$, occupying no processing time on M_1; symmetrically, separators have $b_{s_k} = 0$, occupying no time on M_2. The total M_1 work is $\sum a_j = mB$, the total M_2 work is $\sum b_j = mB$, and the threshold is $Y = 3mB$.

Suppose a 3-Partition exists: There is a partition of the $3m$ elements into m triples $G_1, \ldots, G_m$ with $\sum_{J_j \in G_k} x_j = B$ for each k. Arrange the jobs in the order $G_1, S_1, G_2, S_2, \ldots, G_m, S_m$. Since $a_j = 0$, element jobs placed before S_k complete M_1 at time $(k - 1)B$; each S_k occupies M_1 during $[(k - 1)B, kB]$, so the M_1 timeline is $S_1[0, B], S_2[B, 2B], \ldots, S_m[(m - 1)B, mB]$. On M_2, G_1 finishes at B, S_1 passes instantly at

B, G_2 starts at B and finishes at $2B$, and inductively S_k finishes M_2 at kB with $C_{s_k} = kB$. Consequently

$$w_{s_k} C_{s_k} = \frac{3m}{k} \cdot kB = 3mB = Y, \qquad \max w_j C_j \le \max\{\, 3mB,\ mB \,\} = Y,$$

so $WC_{\max} = Y$.

Conversely, suppose there exists a schedule π with $WC_{\max} \le Y = 3mB$. For each separator S_j, $w_{s_j} C_{s_j} \le Y$ implies $C_{s_j} \le jB$. Each S_j requires B time on M_1; if S_j occupied position p among the separators on M_1 with $p > j$, its M_1 completion would be at least $pB > jB$, contradicting $C_{s_j} \le jB$. Hence $S_1, \dots, S_m$ appear in this order on M_1, and their M_1 completion times are $B, 2B, \dots, mB$.

Define G_k as the set of element jobs placed between S_{k-1} and S_k in π (with S_0 denoting the schedule start), and let G_{m+1} be those after S_m. Set $s_k = \sum_{J_j \in G_k} x_j$, so $\sum_{k=1}^{m+1} s_k = \sum_{i=1}^{3m} x_i = mB$. Jobs in G_k have $a_j = 0$ and complete M_1 at time $(k-1)B$. Let F_k be the M_2 completion time of the last job of G_k (with $F_0 = 0$). Since S_{k-1} passes M_2 instantly at F_{k-1} and all jobs of G_k become M_1-ready at time $(k-1)B$, the first job of G_k starts M_2 at $\max\{(k-1)B,\ F_{k-1}\}$, yielding

$$F_k = \max\{\, (k-1)B,\ F_{k-1} \,\} + s_k, \qquad k = 1, \dots, m.$$

The separator S_k finishes M_2 at $C_{s_k} = \max\{\, kB,\ F_k \,\}$. From $C_{s_k} \le kB$ we obtain $F_k \le kB$ for all k.

We show $s_k \le B$ for all k by induction. For $k = 1$, $F_1 = s_1$ and $F_1 \le B$ imply $s_1 \le B$. Assume $F_{k-1} \le (k-1)B$. Then $F_k = \max\{(k-1)B,\ F_{k-1}\} + s_k \le kB$. If $F_{k-1} \ge (k-1)B$, then $F_k = F_{k-1} + s_k \le kB$, which together with $F_{k-1} \ge (k-1)B$ gives $s_k \le B$. If $F_{k-1} < (k-1)B$, then $F_k = (k-1)B + s_k \le kB$, again yielding $s_k \le B$.

Since $\sum_{k=1}^{m+1} s_k = mB$ and $s_k \le B$ for $k = 1, \dots, m$, we must have $s_k = B$ for every $k = 1, \dots, m$ and $s_{m+1} = 0$; all element jobs therefore precede S_m.

Each G_k satisfies $\sum_{J_j \in G_k} x_j = B$ with $B/4 < x_i < B/2$. Fewer than three elements sum to at most $2 \cdot (B/2 - \varepsilon) < B$, while four or more sum to at least $4 \cdot (B/4 + \varepsilon) > B$. Hence $|G_k| = 3$ for every k, and $\{G_1, \dots, G_m\}$ is a partition of the $3m$ elements into m triples each summing to B, a valid 3-Partition.

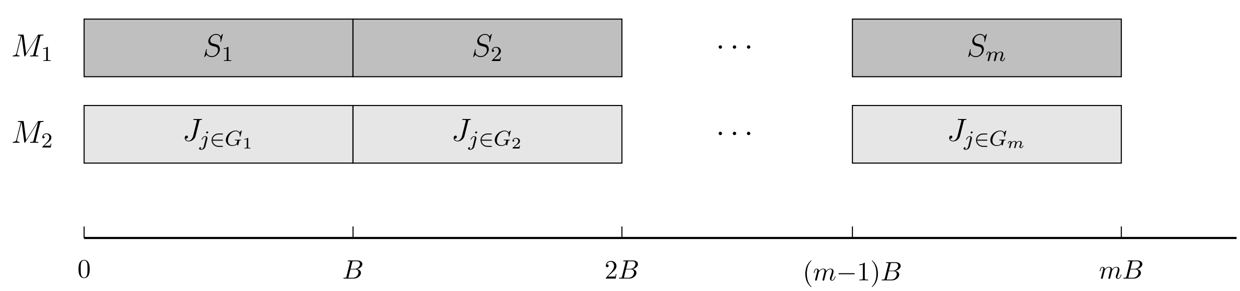

Figure 3-2: 3-Partition reduction schedule.

Thus $F2 \parallel WC_{\max}$ is strongly NP-hard. $\square$

4 $F2 \parallel WC_{\max}$ for the number of weights is constant

In this chapter, we will propose an idea to guide the dynamic programming algorithm for solving this problem, and present the time complexity analysis of the algorithm.

When the number of distinct weights K is bounded by a constant, the structural properties established in Section 2 enable a polynomial-time dynamic programming algorithm. The initial job sequence follows Johnson's rule; the block partition and all subsequent steps are defined with respect to this Johnson order. We first describe the block partition and group logic underlying our approach, then present the DP formulation, and finally give the complete algorithm and its complexity analysis.

4.1 Algorithm Properties and Dynamic Programming

We now develop a dynamic programming algorithm that processes jobs sequentially in Johnson order and assigns each job to one of the K blocks $\mathcal{B}_1, \dots, \mathcal{B}_K$.

Lemma 2.1 provides the structural basis: it guarantees that, given any assignment of jobs to the K blocks, sequencing the jobs within each block $\mathcal{B}_i$ according to Johnson's rule and concatenating the blocks in non-increasing weight order yields a schedule whose makespan is no larger than that of any other schedule respecting the same assignment. We denote by $\mathscr{H}$ the set of states maintained during the computation; each state summarizes the aggregate parameters of all K blocks after a prefix of jobs has been processed. A state is a $4K$-tuple

$$(C_1, \dots, C_K;\ A_1, \dots, A_K;\ B_1, \dots, B_K;\ L_1, \dots, L_K) \in \mathscr{H}$$

for $\mathcal{B}_i$, $i = 1, \dots, K$:

- A_i — total processing time on machine M_1 of jobs in block $\mathcal{B}_i$;
- B_i — total processing time on machine M_2 of jobs in block $\mathcal{B}_i$;
- C_i — makespan of block $\mathcal{B}_i$ when the jobs assigned to it are scheduled in isolation;
- L_i — weight of the last job in block $\mathcal{B}_i$.

The initial state set contains a single state with all components set to zero: $\mathscr{H}^{(0)} = \{(0, \dots, 0)\}$.

State Transition. For $j = 1.....n$ perform the following computations. Set $Y_i \leftarrow \emptyset, \quad i = 1, \ldots, k$. For each tuple $(C_1, \ldots, C_K;\ A_1, \ldots, A_K;\ B_1, \ldots, B_K;\ L_1, \ldots, L_K) \in \mathscr{H}^{(j-1)}$ and $i = 1.....K$, if $w_j \leqslant L_i$ do the following.

$$\begin{aligned} Y_i \leftarrow Y_i \cup \Big\{ \big(C_1, \ldots, C_{i-1}, \max\{C_i + b_j,\ A_i + a_j + b_j\}, C_{i+1}, \ldots, C_K, \\ A_1, \ldots, A_{i-1},\ A_i + a_j,\ A_{i+1}, \ldots, A_K, \\ B_1, \ldots, B_{i-1},\ B_i + b_j,\ B_{i+1}, \ldots, B_K\big) \Big\} \end{aligned}$$

Merge $i = 1, ..., K$, to obtain $\mathscr{H}^{(j)}$. If we get the same tuples during the merge, store only one of them.

Once all jobs are assigned and each block $\mathcal{B}_i$ is summarized by its parameters (A_i, B_i, C_i), the overall makespan $C_{\max}$ is obtained by concatenating the blocks in order of increasing index. The procedure is given below:

In the update of T_B, the term $T_A - A_i$ is the start time of $\mathcal{B}_i$ on M_1, so $T_A - A_i + C_i$ is the completion time of $\mathcal{B}_i$ on M_2 when M_2 is free, while $T_B + B_i$ accounts for the case where M_2 is the bottleneck.

Algorithm 1 Block concatenation

Input: Block parameters (A_i, B_i, C_i) for $i = 1, \ldots, K$.
Output: Completion time C on M_2.
$T_{A_0} \leftarrow 0, \quad T_{B_0} \leftarrow 0$
for $i = 1$ to K **do**
    $T_{A_i} \leftarrow T_{A_{i-1}} + A_i$
    $T_{B_i} \leftarrow \max\{\, T_{A_{i-1}} + C_i,\ T_{B_{i-1}} + B_i \,\}$
end for
$C \leftarrow T_{B_K}$
return C

Algorithm 2 DP for $F2 \parallel WC_{\max}$

Input: Jobs $\mathcal{J}$ with (a_j, b_j, w_j) in Johnson order; block count K.
Output: The optimal value $Z^* = \min_S \max_i \, L_i \cdot T_B^{(i)}$.
$\mathscr{H}^{(0)} \leftarrow \{(0, \ldots, 0)\}$
for $j = 1$ to n **do**
  $Y_i \leftarrow \emptyset$ for $i = 1, \ldots, K$
  for each $(\{C_i, A_i, B_i, L_i\}_{i=1}^K) \in \mathscr{H}^{(j-1)}$ **do**
    for $i = 1$ to K **do**
      if $w_j \le L_i$ **then**
        $C_i' \leftarrow \max\{\, C_i + b_j,\ A_i + a_j + b_j \,\}$
        $A_i' \leftarrow A_i + a_j, \quad B_i' \leftarrow B_i + b_j$
        Add $(\ldots, C_i', \ldots; \ldots, A_i', \ldots; \ldots, B_i', \ldots; \ldots, L_i, \ldots)$ to Y_i
      end if
    end for
  end for
  $\mathscr{H}^{(j)} \leftarrow \textsc{Merge}(Y_1, \ldots, Y_K)$
end for
$Z^* \leftarrow +\infty$
for each state $S = \{C_i, A_i, B_i, L_i\}_{i=1}^K \in \mathscr{H}^{(n)}$ **do**
  for $i = 1$ to K **do**
    $C(S) \leftarrow \textsc{Algorithm 1}\big((A_1, B_1, C_1), \ldots, (A_i, B_i, C_i)\big)$
  end for
  $WC_{\max} \leftarrow \max_{i=1,\ldots,K} \, L_i \cdot C(S),$
  $Z^* \leftarrow \min\{\, Z^*,\ WC_{\max} \,\}$
end for
return Z^*

Theorem 4.1. *Algorithm 2 solves $F2 \parallel WC_{\max}$ optimally in $O\big(n \cdot K \cdot |\mathscr{H}|_{\max}\big)$ time, where $|\mathscr{H}|_{\max} \le K^K \cdot V^{3K-2}$, $V = \max\big\{\sum_{j=1}^n a_j,\ \sum_{j=1}^n b_j\big\}$. For constant K, this is $O(n \cdot V^{3K-2})$.*

Proof. Each state in $\mathscr{H}^{(j)}$ is a $4K$-tuple $(C_1, \ldots, C_K;\ A_1, \ldots, A_K;\ B_1, \ldots, B_K;\ L_1, \ldots, L_K)$. We bound the number of distinct values each component can attain.

The quantity A_i is the total M_1 processing time of jobs assigned to block $\mathcal{B}_i$, hence $A_i \in [0, V]$. Since the job sequence is fixed and the j jobs under consideration form a prefix of the Johnson order, $\sum_{i=1}^K A_i$ equals the sum of a_j over that prefix. Consequently at most $K - 1$ of the A_i are independent, contributing at most V^{K-1} distinct A-vectors. Symmetrically, $\sum_{i=1}^K B_i$ is the prefix sum of the b_j's, so the B_i components likewise contribute at most V^{K-1} distinct combinations.

The quantity C_i is the makespan of $\mathcal{B}_i$ when scheduled in isolation by Johnson's rule. Since $C_i \le A_i + B_i \le 2V$, the C-vector contributes at most $(2V)^K$ distinct values. The weight L_i is the weight of the last job in $\mathcal{B}_i$ and takes values from $\{w^{(1)}, \dots, w^{(K)}\}$, yielding at most K^K distinct L-vectors.

Multiplying these bounds yields

$$|\mathscr{H}^{(j)}| \le K^K \cdot (2V)^K \cdot V^{2K-2} = O\big(K^K \cdot 2^K \cdot V^{3K-2}\big).$$

In iteration j, the algorithm takes each state in $\mathscr{H}^{(j-1)}$, tries every block $i \in \{1, \dots, K\}$, computes the updated tuple in $O(1)$ time, and inserts it into Y_i. The merge step collects the K sets $Y_1, \dots, Y_K$ and removes duplicates in time proportional to the total number of generated tuples. Hence iteration j costs $O(K \cdot |\mathscr{H}^{(j-1)}|)$, and summing over all n iterations gives $O(n \cdot K \cdot |\mathscr{H}|_{\max})$.

The final concatenation via Algorithm 1 evaluates each state in $\mathscr{H}^{(n)}$ in $O(K)$ time, which is dominated by the DP phase. When K is fixed, $K^K \cdot 2^K$ is constant, giving $O(n \cdot V^{3K-2})$. $\square$

4.2 FPTAS

5 Conclusion

References

Baker, K. R. (1974). *Introduction to Sequencing and Scheduling*. Wiley, New York.

Hall, L. A. (1994). A polynomial approximation scheme for a constrained flow-shop scheduling problem. *Mathematics of Operations Research*, 19(1):68–85.

Johnson, S. M. (1954). Optimal two- and three-stage production schedules with setup times included. *Naval Research Logistics Quarterly*, 1(1):61–68.